Database Schema — The Five-Step Spine

Postgres, written for Supabase but portable. Twelve tables covering signup → add client → intake form → view answers → subscription with alert. Nothing beyond that, deliberately.

The shape

Twelve tables for the five-step spine

Indentation is the foreign-key chain; every table under workspaces carries workspace_id

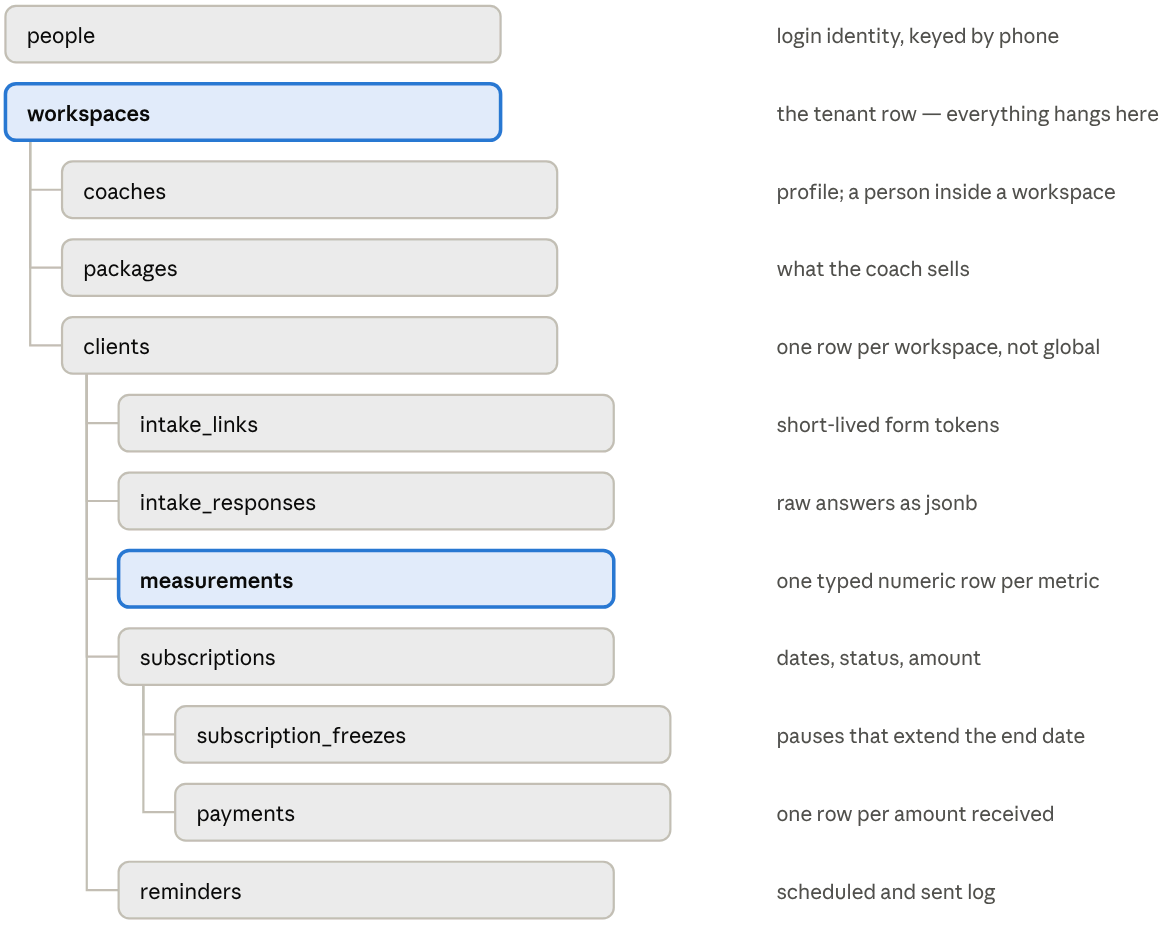


table tree · foreign-key chain

Four decisions are baked into this and all four are expensive to reverse later:

1. **`workspaces` is the tenant, not `coaches`.** Today one coach equals one workspace, so it looks redundant. The day a studio signs up with three coaches sharing clients, you add rows instead of rewriting every query and policy.

2. **`people` is separate from `clients`.** A person is a login keyed by phone; a client is that person's relationship with one workspace. This is what lets someone train with two coaches without you duplicating logins — and it lets a coach add a client who has never signed up at all (`clients.person_id` stays null).
3. **`measurements` is long-format, not columns.** One row per metric per date, rather than a wide table with `weight`, `waist`, `chest`. Adding a new metric becomes a row, not a migration, and charting is a single query.
4. **`intake_responses` keeps the raw jsonb, but numbers are also copied into `measurements`.** The jsonb is the audit trail of what the client actually typed; the typed rows are what the charts read. Don't make the charts parse json.

Identity and tenancy

```
create extension if not exists "pgcrypto";

-- A human being with a login. Global, not scoped to any workspace.
create table people (
  id            uuid primary key default gen_random_uuid(),
  phone         text not null unique,          -- E.164: +201012345678
  full_name     text not null,
  email         text unique,                   -- optional fallback login
  auth_user_id  uuid unique,                   -- maps to auth.users on
Supabase
  created_at    timestamptz not null default now()
);

-- The tenant. One per coach today; one per studio later.
create table workspaces (
  id            uuid primary key default gen_random_uuid(),
  brand_name    text not null,                 -- what clients see
  slug          text not null unique,          -- ahmed.yourapp.com
  logo_url      text,
  currency      char(3) not null default 'EGP',
  unit_system   text not null default 'metric'
                  check (unit_system in ('metric','imperial')),
  expiry_alert_days int not null default 7,
  created_at    timestamptz not null default now()
);

-- Membership: this person is a coach in this workspace.
```

```
create table coaches (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  person_id     uuid not null references people(id) on delete restrict,
  role          text not null default 'owner'
                  check (role in ('owner','coach','assistant')),
  specialties   text[] not null default '{}',
  bio           text,
  instagram     text,
  created_at    timestamptz not null default now(),
  unique (workspace_id, person_id)
);

create index on coaches (person_id);
```

Why `people` is split from `coaches`

The login belongs to the human; the role belongs to the workspace. A coach who later joins a studio, or a coach who is also someone else's client, costs you nothing with this split and costs you a migration without it.

`role` is there now because the column is free today and painful to backfill once you have real data. You simply don't expose anything but `owner` in the UI yet.

Signup writes three rows

Signup is one transaction: insert a `person`, insert a `workspace`, insert a `coach` joining them. Wrap it in a function so it can't half-succeed:

```
create function sign_up_coach(
  p_phone text, p_name text, p_brand text, p_auth_user uuid
) returns uuid language plpgsql security definer as $$
declare v_person uuid; v_workspace uuid;
begin
  insert into people (phone, full_name, auth_user_id)
    values (p_phone, p_name, p_auth_user)
    returning id into v_person;

  insert into workspaces (brand_name, slug)
    values (p_brand, slugify(p_brand))
    returning id into v_workspace;

  insert into coaches (workspace_id, person_id, role)
    values (v_workspace, v_person, 'owner');

  return v_workspace;
end $$;
```

Clients and intake

```
create table clients (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  person_id     uuid references people(id) on delete set null,  -- null until
they sign up
  full_name     text not null,
  phone         text not null,
  intake_status text not null default 'pending'
                  check (intake_status in ('pending','submitted','skipped')),
  deleted_at    timestamptz,                    -- soft delete
  created_at    timestamptz not null default now(),
  unique (workspace_id, phone)
);

create index on clients (workspace_id) where deleted_at is null;
create index on clients (person_id);
```

`phone` is stored on the client row as well as on `people`, because the coach types a number before that person exists in your system. When the client later signs up, you match on phone and backfill `person_id`.

`deleted_at` instead of a real delete: a coach who removes a client by mistake would otherwise lose that client's entire payment history.

Intake links

```
create table intake_links (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  token         text not null unique,          -- 8 chars, url-safe, random
  expires_at    timestamptz not null default now() + interval '14 days',
  submitted_at  timestamptz,                   -- null = still open
  created_at    timestamptz not null default now()
);

create index on intake_links (token) where submitted_at is null;
```

Generate `token` with a CSPRNG, not a sequence or a hash of the client id. Eight url-safe characters gives you ~47 bits — plenty when links also expire, and short enough to look trustworthy in a WhatsApp message.

The partial index means your hot lookup (open link by token) touches a small index even when you have a million expired rows.

Responses

```
create table intake_responses (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  link_id       uuid references intake_links(id) on delete set null,
  answers       jsonb not null,                -- raw, exactly as submitted
  form_version  text not null default 'default_v1',
  submitted_by  text not null default 'client'
                  check (submitted_by in ('client','coach')),
  submitted_at  timestamptz not null default now()
);

create index on intake_responses (client_id, submitted_at desc);
create index on intake_responses using gin (answers);
```

`form_version` matters more than it looks. When you ship the form builder later, every stored response needs to say which question set produced it — otherwise old answers become uninterpretable. Stamping a version now costs one column; adding it after you have 10,000 responses means guessing.

`submitted_by` covers the case from the edge-case list: the client who refuses to fill a form and the coach fills it for him.

Measurements

```
create table measurements (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  metric        text not null,                -- 'weight', 'waist',
'sleep_quality'
  value         numeric(8,2) not null,
  unit          text not null,                -- 'kg', 'cm', 'score'
  measured_at   date not null,
  source        text not null default 'checkin'
                  check (source in ('intake','checkin','coach')),
  created_at    timestamptz not null default now(),
  unique (client_id, metric, measured_at)
);

create index on measurements (client_id, metric, measured_at desc);
```

Why long-format, concretely

A wide table (`weight`, `waist`, `chest`, `arm`, `sleep`…) looks tidier and costs you on all three axes that matter:

Need	Wide table	Long format
Add a new metric	Migration + code change	Insert a row
Chart any metric	A different query per column	One query, parameterised
Client measured weight today, waist next week	Sparse nulls everywhere	Just fewer rows

Need	Wide table	Long format
Coach-specific custom metric	Impossible without per-tenant columns	Works already

The `unique (client_id, metric, measured_at)` constraint is doing real work: it makes a re-submitted check-in an upsert instead of a duplicate point on the chart.

Keep the metric keys in code, not in a table

```
-- a lookup table for display only, not a foreign key
create table metric_defs (
  metric        text primary key,
  label_ar      text not null,
  label_en      text not null,
  default_unit  text not null,
  kind          text not null check (kind in ('body','habit','score'))
);
```

Don't make `measurements.metric` a foreign key to this. The moment a coach wants to track something you didn't anticipate, a hard FK turns a feature into a deployment. Validate the key in your application layer and treat this table as labels.

Copying intake numbers into measurements

The intake form's weight, height and measurements should land here as the client's first data point, with `source = 'intake'`. Do this in the same transaction that writes `intake_responses` — if it's a separate job it will silently fail and you'll have clients whose charts start a month late.

Packages, subscriptions and money

```
create table packages (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  name          text not null,                -- 'باقة 3 شهور'
  duration_days int not null check (duration_days > 0),
  price_minor   bigint not null check (price_minor >= 0),  -- piastres, not
pounds
  currency      char(3) not null default 'EGP',
  is_active     boolean not null default true,
  created_at    timestamptz not null default now()
);
```

Store money as integer minor units. `price_minor = 150000` is EGP 1,500.00. Floats accumulate rounding errors, and `numeric` invites someone to divide by three. Format for display only at the edge.

Packages are never hard-deleted — `is_active = false` — because old subscriptions point at them.

Subscriptions

```
create table subscriptions (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  package_id    uuid references packages(id) on delete restrict,

  -- snapshot at purchase time, not a lookup
  package_name  text not null,
  price_minor   bigint not null,
  currency      char(3) not null,

  starts_on     date not null,
  ends_on       date not null,
  status        text not null default 'active'
                  check (status in ('active','frozen','ended','cancelled')),
  created_at    timestamptz not null default now(),
  check (ends_on >= starts_on)
);
```

```
create index on subscriptions (workspace_id, status, ends_on);
create index on subscriptions (client_id, starts_on desc);

-- at most one live subscription per client
create unique index one_live_sub_per_client
  on subscriptions (client_id)
  where status in ('active','frozen');
```

The snapshot columns are the important part. If the coach raises his three-month package from 1,500 to 1,800 next month, every historical subscription must still show what was actually charged. Joining to `packages` for the price would silently rewrite your revenue history.

That partial unique index enforces the edge case about one client with two plans — the database refuses it rather than your application remembering to check.

Why `status` has four values, not five

The flows document lists five states. Only four are stored:

State	Stored?	How you get it
active	✅	`status = 'active'`
frozen	✅	`status = 'frozen'`
ended	✅	`status = 'ended'`
cancelled	✅	`status = 'cancelled'`
expiring soon	❌ derived	`ends_on <= current_date + workspace.expiry_alert_days`
payment overdue	❌ derived	sum of payments < `price_minor`

Derived states must never be stored. A stored "expiring soon" is wrong the next morning and needs a job to maintain it; a computed one is always right.

Freezes

```
create table subscription_freezes (
  id              uuid primary key default gen_random_uuid(),
  workspace_id    uuid not null references workspaces(id) on delete cascade,
  subscription_id uuid not null references subscriptions(id) on delete
cascade,
  frozen_on       date not null,
  resumed_on      date,                        -- null = still frozen
  reason          text,
  created_at      timestamptz not null default now()
);

create unique index one_open_freeze
  on subscription_freezes (subscription_id)
  where resumed_on is null;
```

On resume: set `resumed_on`, add `(resumed_on - frozen_on)` days to `subscriptions.ends_on`, set status back to `active`. Keeping the freeze rows rather than only adjusting the date means you can always explain to a client why his end date moved.

Payments

```
create table payments (
  id              uuid primary key default gen_random_uuid(),
  workspace_id    uuid not null references workspaces(id) on delete cascade,
  subscription_id uuid not null references subscriptions(id) on delete
restrict,
  amount_minor    bigint not null check (amount_minor > 0),
  currency        char(3) not null,
  method          text not null
                    check (method in
('instapay','wallet','cash','bank','card','other')),
  paid_on         date not null,
  receipt_url     text,
  note            text,
  recorded_by     uuid references coaches(id) on delete set null,
  created_at      timestamptz not null default now()
);

create index on payments (workspace_id, paid_on desc);
create index on payments (subscription_id);
```

One row per amount received, not a `paid / partial` flag on the subscription. Partial payment is normal in this market, and "he paid 800 now and 700 next week" has to be two rows. Payment status is then a sum, computed:

```
create view subscription_balances as
select s.id as subscription_id,
       s.workspace_id,
       s.price_minor,
       coalesce(sum(p.amount_minor), 0) as paid_minor,
       s.price_minor - coalesce(sum(p.amount_minor), 0) as due_minor
from subscriptions s
left join payments p on p.subscription_id = s.id
group by s.id;
```

Reminders

```
create table reminders (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid references clients(id) on delete cascade,
  subject_type  text not null
                  check (subject_type in
('subscription','intake','checkin')),
  subject_id    uuid not null,                -- the subscription / link
being nagged about
  kind          text not null
                  check (kind in
('expiring_soon','expired','payment_due','intake_nudge')),
  audience      text not null check (audience in ('coach','client')),
  channel       text not null check (channel in
('whatsapp','inapp','email')),
  scheduled_for timestamptz not null,
  sent_at       timestamptz,
  failed_reason text,
  created_at    timestamptz not null default now(),
  unique (subject_type, subject_id, kind, audience)
);

create index on reminders (scheduled_for) where sent_at is null;
```

The one constraint that matters

`unique (subject_type, subject_id, kind, audience)` is what stops you from WhatsApping a client six times that his subscription expires. Without it, any job that runs more than once — a retry, a redeploy, two workers — double-sends. In this market a duplicate WhatsApp message to a paying client is a real cost, not a cosmetic bug.

Rows are created, not discovered

Two ways to build this, and the difference shows up in month three:

- **Query-on-the-fly:** a cron job each morning asks "which subscriptions expire in 7 days?" and sends. Simple, but you have no record of what was sent, no retry, and no way to answer "did he actually get the message?"
- **Materialise the reminder:** when a subscription is created, insert its reminder rows immediately with their `scheduled_for` dates. The sender just drains `where sent_at is null and scheduled_for <= now()`.

Take the second. It gives you the audit trail, idempotency through that unique constraint, and a trivially simple sender.

When `ends_on` changes — a freeze, a manual edit — delete the unsent reminders for that subscription and re-create them. Unsent only; never touch rows that already went out.

The sender

```
-- claim a batch atomically so two workers can't send the same row
update reminders
set sent_at = now()
where id in (
  select id from reminders
  where sent_at is null and scheduled_for <= now()
  order by scheduled_for
  limit 100
  for update skip locked
)
returning *;
```

`for update skip locked` is the whole trick. It lets you run several senders without locking each other out or sending duplicates. Mark `sent_at` when you claim, and write `failed_reason` if the send fails — pessimistic beats sending twice.

Row-level security

This is the part that makes multi-tenancy real. Without it, one missing `where workspace_id = ...` in one query leaks another coach's clients.

The helper

```
-- which workspaces does the current logged-in user coach in?
create or replace function auth_workspace_ids()
returns setof uuid
language sql stable security definer
set search_path = public
as $$
  select c.workspace_id
  from coaches c
  join people p on p.id = c.person_id
  where p.auth_user_id = auth.uid()
$$;
```

`stable` lets Postgres cache it within a statement instead of re-running per row. On a table with thousands of rows that is the difference between a fast policy and an unusable one.

Policies

```
alter table workspaces          enable row level security;
alter table coaches             enable row level security;
alter table clients             enable row level security;
alter table intake_links        enable row level security;
alter table intake_responses    enable row level security;
alter table measurements        enable row level security;
alter table packages            enable row level security;
alter table subscriptions       enable row level security;
alter table subscription_freezes enable row level security;
alter table payments            enable row level security;
alter table reminders           enable row level security;

-- every workspace-scoped table gets the same shape
create policy tenant_rw on clients
  for all
  using      (workspace_id in (select auth_workspace_ids()))
  with check (workspace_id in (select auth_workspace_ids()));
```

```
-- repeat verbatim for: intake_links, intake_responses, measurements,
-- packages, subscriptions, subscription_freezes, payments, reminders

create policy tenant_rw on workspaces
  for all
  using      (id in (select auth_workspace_ids()))
  with check (id in (select auth_workspace_ids()));
```

`with check` is as important as `using`. `using` controls what you can read; `with check` stops someone writing a row into *another* workspace. Policies that only set `using` are a common and quiet hole.

The client's own access

A client logs in too, and must see his own data without seeing the workspace. That's a second policy, not a relaxation of the first:

```
create policy client_reads_self on measurements
  for select
  using (client_id in (
    select cl.id from clients cl
    join people p on p.id = cl.person_id
    where p.auth_user_id = auth.uid()
  ));
```

Policies are OR-ed together, so a coach matches the tenant policy and a client matches this one. Give the client `select` only — never `for all`.

The intake form is the exception

The form opens with no login, so RLS can't help: there's no `auth.uid()`. Don't open a policy for it. Instead, submission goes through a `security definer` function that takes the token and does the authorisation itself:

```
create or replace function submit_intake(p_token text, p_answers jsonb)
returns uuid
language plpgsql security definer
set search_path = public
as $$
declare v_link intake_links; v_response uuid;
begin
```

```
  select * into v_link from intake_links
   where token = p_token and submitted_at is null and expires_at > now()
   for update;

  if not found then
    raise exception 'invalid_or_expired_link' using errcode = 'P0001';
  end if;

  insert into intake_responses (workspace_id, client_id, link_id, answers)
    values (v_link.workspace_id, v_link.client_id, v_link.id, p_answers)
    returning id into v_response;

  update intake_links set submitted_at = now() where id = v_link.id;
  update clients set intake_status = 'submitted' where id = v_link.client_id;

  -- copy numeric answers into measurements here, same transaction

  return v_response;
end $$;

revoke all on function submit_intake from public;
grant execute on function submit_intake to anon;
```

The anonymous role gets permission to call this one function and nothing else. The token is the credential, the function is the only door, and everything it touches stays inside one transaction.

Test it before you trust it

Write two coaches and two clients into a seed script, then assert that coach A's session returns zero rows for every one of coach B's tables. Do it as an automated test, not by clicking around. A tenancy leak found by a customer is not recoverable in a market this small.

What's deliberately missing

Everything below belongs to the product but not to the spine. Each one is additive — new tables, no changes to what's above — which is the test for whether deferring it is safe.

Deferred	Tables it will add	Does it touch the spine?
Form builder	`form_templates`, `form_questions`	No — `intake_responses.form_version` already points at it
Workout plans	`exercises`, `plans`, `plan_days`, `plan_exercises`, `sets`	No
Nutrition	`foods`, `food_units`, `meal_plans`, `meals`, `meal_items`, `meal_item_subs`	No
Check-ins	`checkin_schedules`, `checkins`	No — writes into `measurements`, which exists
Platform billing (coaches paying you)	`plans_catalog`, `platform_subscriptions`, `platform_invoices`	No — different concern, keep it separate

The last row is worth a sentence: don't reuse `subscriptions` for the coach's own subscription to your platform. They look alike and diverge fast, and mixing them makes revenue reporting a mess.

Migration order

Run these as separate numbered migrations, not one file. You'll want to replay them.

1. `people`, `workspaces`, `coaches`, plus `sign_up_coach`
2. `clients`
3. `intake_links`, `intake_responses`, plus `submit_intake`
4. `metric_defs`, `measurements`
5. `packages`, `subscriptions`, `subscription_freezes`, `payments`, `subscription_balances`
6. `reminders`
7. RLS: enable on every table, create every policy, grant `submit_intake` to `anon`

Step 7 last and in one migration, so you can review every policy together rather than finding out one table never got RLS enabled.

Two things to do on day one

Enable RLS before you insert a single real row. A table with RLS off is wide open, and "we'll turn it on before launch" is how tenancy bugs ship.

Seed two full workspaces in your dev script. One coach with three clients is not enough to catch isolation bugs — with a single tenant, every broken query still returns the right answer.

Where I'd push back on myself

The `workspaces` / `people` split is the one place this schema is more than the five steps need. A single `coaches` table with `coach_id` everywhere would ship a week sooner.

I'd still take the split, because both are in the "expensive to reverse" category and cost about one extra day now. But if you'd rather have the week, collapse `people` into `coaches` and keep `workspaces` — the tenant table is the one that genuinely cannot be retrofitted.